Private Cloud Storage for a Next.js Site on Firebase App Hosting

Fulllion Creative Works

A step-by-step guide to giving a Next.js site on Firebase App Hosting a private Cloud Storage bucket. The site stores users' files there and hands out short-lived signed links, so files go straight between the user and the bucket without passing through the site.

> **Starting from scratch?** This guide assumes your site already runs on App Hosting. To get there, see Building Firebase Apps in Google Antigravity IDE and Connecting a Custom Domain to Firebase App Hosting.

Throughout, replace:

Placeholder	With
`<project-id>`	your Firebase project's ID
`<project name>`	your project's display name, which the Cloud console shows in place of the ID
`<bucket-name>`	the name you give the bucket in Section 3
`<region>`	your App Hosting backend's region, such as `us-central1`
`<your-domain>`	your site's address, such as `example.com`

Contents

1. How the Pieces Fit Together
2. Find Your Project in the Google Cloud Console
3. Create the Bucket
4. Give the Site Access
5. Connect the Site
6. Limits That Break Large Uploads
7. Costs to Watch
8. Troubleshooting
9. Sources

Before You Start

You'll need:

- **A Next.js site running on Firebase App Hosting.** Note which region its backend is in: Firebase console ▸ **App Hosting** ▸ your backend.

- **The Blaze plan.** Both Cloud Storage and App Hosting need Firebase's pay-as-you-go plan. The quickest check is the bottom of the Firebase console's left sidebar, which shows **Blaze** and **Pay as you go**. For more detail, go to **Project settings** ▸ **Usage and billing**: the **Project cost** card on the **Overview** tab has a **Blaze** badge, and budgets are on the **Account & budgets** tab.
- **The Google account that owns the Firebase project.** Everything here happens in the Google Cloud console, and it only shows projects for the account you're signed in with.
- `firebase-admin` in your project (`npm install firebase-admin`).
- **About an hour,** plus a few minutes of waiting for permissions to take effect.

---

1. How the Pieces Fit Together

Piece	What it is
Bucket	A container for files in Cloud Storage. Yours stays private, so nobody can read or write it without permission.
Object	One stored file. Its name can include slashes, such as `users/abc123/save.bin`, which look like folders.
Service account	The identity your site runs as on App Hosting: `firebase-app-hosting-compute@<project-id>.iam.gserviceaccount.com`. You give *it* permissions, so your code needs no key file.
Signed link	A web address for one file and one action (upload or download) that works for a few minutes. Anyone holding it can do that one thing, and nothing else.

When a user saves a file:

```
User's app ──"I want to upload a 12 MB file"──────────▶ Your site
Your site  ──checks the user, signs a 15-minute link──▶ User's app
User's app ──uploads the file, straight to────────────▶ Your bucket
```

Downloads work the same way, with a link signed for reading. The file never passes through your site, which matters for size limits (Section 6) and for cost.

---

2. Find Your Project in the Google Cloud Console

Every Firebase project is also a Google Cloud project, with the same project ID. It can still look missing from the Cloud console, almost always because the console is signed in to a different Google account.

1. Open this address, with your project ID in place of `<project-id>`:

   ```
   https://console.cloud.google.com/storage/browser?project=<project-id>
   ```

2. If it says you don't have access, or shows a different project, click your profile picture (top right) and switch to the account that owns the Firebase project.
3. To pick the project from the list instead, open the project picker at the top of the page and choose the **All** tab. The **Recent** tab only shows projects you've opened in the Cloud console before.

The picker and the top bar show your project's display name (`<project name>`), not its ID, so look for the name you gave it in Firebase.

---

3. Create the Bucket

Go to **Cloud Storage ▸ Buckets** and click **Create**. The **Create a bucket** page has five steps, covered below. Each ends with a **Continue** button, and a finished step collapses into a short summary.

3.1 Get started

Under **Pick a globally unique, permanent name**, enter `<bucket-name>`:

- The name must be **unique across all of Google Cloud**, not just your project, so a plain name like `saves` will be taken.
- It's **permanent**, and it shows up in every link you sign. The form's own tip says not to include sensitive information.
- Lowercase letters, numbers, dashes, underscores and dots are allowed.

Labels (optional) can stay empty. Click **Continue**.

3.2 Choose where to store your data

The form starts on **Location type: Multi-region**, with **us (multiple regions in United States)** selected. Change it: choose **Region**, then pick **the same region as your App Hosting backend** (`<region>`).

The **Good to know ▸ Location pricing** panel shows the price of whatever you've selected. For the default US multi-region, it showed $0.026 per GB-month, plus **$0.020 per GB written** for replication.

Multi-region can look like the safer choice for reliability, but it adds little for a site like this:

	Region (recommended)	Multi-region
Where the data lives	One region, such as `us-central1`	A large area, such as the US
Uptime promise (SLA)	99.9% a month	99.95% a month
Durability	99.999999999% (eleven nines)	The same
Storage price	Lowest	Higher
Writing data	No extra charge	About $0.02 per GiB written (US and EU), to copy it across regions

Why one region is the better fit:

- **Your site runs in one region anyway.** If that region goes down, your site is down too, so a multi-region bucket would only be reachable by a site that isn't there.
- **"Multi-region" isn't worldwide.** The US multi-region covers the US only. It doesn't speed anything up for users in other countries.

- **Durability is the same either way.** Your files are just as safe in a single region. Multi-region only adds a little uptime.
- **The same region is cheapest to use.** Your site and bucket talk to each other within one region.

> **The location is effectively permanent.** Google only offers bucket relocation through its paid Storage Intelligence service. Otherwise, changing it later means creating a new bucket and copying everything across.

3.3 Choose how to store your data

- **Storage class: Standard.** The cheaper classes charge for each read and for deleting files early, which suits archives, not files people use.
- **Enable Hierarchical namespace on this bucket:** leave it **unticked**. The form calls it a permanent option, and it's built for analytics workloads.
- **Enable Rapid Cache:** leave it **unticked**. It adds paid, SSD-backed read caches for data that many machines read at once. The form also says it helps avoid multi-region data transfer fees, which a single-region bucket doesn't have.

3.4 Choose how to control access to objects

> **Open this step, even if its summary looks right.** The collapsed summary can say **Public access prevention: On** while the checkbox inside is unticked.

- **Prevent public access: tick "Enforce public access prevention on this bucket".** This stops the bucket or any file in it from ever being made public. Signed links still work.
- **Access control: Uniform.** Permissions apply to the whole bucket, not file by file. The form notes it becomes permanent after 90 days, which is fine here. You won't need **Fine-grained**.
- **IP filtering (optional):** leave it alone, without clicking **Configure**. Your users will upload from anywhere, so limiting addresses would block them.

3.5 Choose how to protect object data

These settings sit under **Data protection**.

- **Soft delete policy (For data recovery): turn it off.** It's ticked by default, with **Use default retention duration** (7 days). Untick it, or choose **Set custom retention duration** and enter `0`, which the form notes disables soft delete. Soft delete keeps every deleted file for those 7 days, and you pay to store it all that time. If your app keeps its own history of files, soft delete adds nothing but cost, and frequent saves add up fast.
- **Object versioning (For version control):** leave it **unticked,** for the same reason.
- **Retention (For compliance):** leave it **unticked.** A retention policy stops anyone deleting files before a set time, including your own app.
- **Data Encryption:** leave this section as it is, with Google-managed encryption. Your files are still encrypted. Google just manages the keys.

Click **Create**.

3.6 Check it from outside

From any terminal, ask for the bucket without signing in:

```
curl https://storage.googleapis.com/<bucket-name>
```

On Windows PowerShell, type `curl.exe` instead of `curl`.

Answer contains	Means
`AccessDenied`	The bucket exists and is private. This is what you want.
`NoSuchBucket`	The name is wrong, or the bucket wasn't created.
A list of files	The bucket is public. Go back to 3.4 and enforce public access prevention.

A private bucket answers with something like this:

```
<Error><Code>AccessDenied</Code><Message>Anonymous caller does not have storage.objects.list
access…</Message></Error>
```

It's worth running this once before you create the bucket too. The answer changes from `NoSuchBucket` to `AccessDenied`, which confirms you're checking the right name.

---

4. Give the Site Access

Your site runs as the App Hosting service account:

```
firebase-app-hosting-compute@<project-id>.iam.gserviceaccount.com
```

It needs two permissions: one to use the bucket, and one to sign links.

4.1 Let it read and write files

1. Go to **Cloud Storage** ▸ **Buckets** ▸ your bucket ▸ the **Permissions** tab.
2. Click **Grant access**. Some versions of the page call it **Add principal**.
3. Under **New principals**, enter the service account's address.
4. Under **Role**, choose **Storage Object Admin**.
5. Click **Save**.

Don't choose **Storage Admin**. It also lets the account change the bucket's settings, or delete it.

4.2 Let it sign links

Signing a link needs a private key. Rather than giving your site a key file, you let the service account ask Google to sign on its behalf. For that, it needs the **Service Account Token Creator** role **on itself**.

1. Go to **IAM & Admin** ▸ **Service Accounts**.

2. Click the `firebase-app-hosting-compute@…` account. Its page is titled **Firebase App Hosting compute service account**, with the tabs **Details**, **Permissions**, **Keys**, **Metrics**, **Logs** and **Principals with access**.
3. Open the **Principals with access** tab.
4. Click **Grant access**.
5. Under **New principals**, enter **the same account's own address**.
6. Under **Role**, choose **Service Account Token Creator**.
7. Click **Save**. A message confirms **Policy updated**, and warns that changes can take a few minutes to become active.

> **The trap: the Permissions tab.** It's the obvious place to look, but it's the wrong one. It has three parts:
>
> - **Manage service account permissions**, with a **Manage access** button. This opens a panel titled **Edit access to "** `<project name>` **"**, which lists the account's roles across the **whole project**: **Developer Connect Read Token Accessor**, **Firebase Admin SDK Administrator Service Agent** and **Firebase App Hosting Compute Runner**.
> - **View service account permissions**, with **Run Policy Analyzer**.
> - An info box under **Principals with access to this service account**, which links to the right tab.
>
> In the **Edit access** panel:
>
> - **Don't add the role there.** Granted at the project level, it would let the account act as every service account in the project.
> - **Never remove the roles listed there.** Your site needs them to run. If you open the panel by mistake, click **Cancel**.
>
> Button names vary between screens and over time (**Grant access**, **Add principal**, **Manage access**). Go by where you are instead: the **Principals with access** tab of the service account's own page.

4.3 Turn on the signing API

1. Go to **APIs & Services** ▸ **Library**, or open `https://console.cloud.google.com/apis/library/iamcredentials.googleapis.com?project=<project-id>`.
2. Search for **IAM Service Account Credentials API** and open it. Its **Product details** page is labelled **Google Enterprise API**, with the service name `iamcredentials.googleapis.com`.
3. If it shows **API Enabled**, with **Manage** and **Try this API** buttons, it's already on, and there's nothing to do. Otherwise, click **Enable**.

4.4 Wait a few minutes

Permission changes can take a few minutes to reach every Google server, as the **Policy updated** message says. If signing fails right after this section, wait five minutes and try again before changing anything.

---

5. Connect the Site

5.1 Name the bucket in apphosting.yaml

Add the bucket name as a plain environment variable. No key file or secret is needed, because the site runs as the service account you just set up.

```
env:
  - variable: STORAGE_BUCKET
    value: <bucket-name>
    availability:
      - RUNTIME
```

5.2 Open the bucket in your code

Keep this on the server only. In the App Router, that means using it only in route handlers and server code.

```
// lib/storage.ts
import { getApps, initializeApp } from 'firebase-admin/app';
import { getStorage } from 'firebase-admin/storage';

const app = getApps()[0] ?? initializeApp();

export function bucket() {
  const name = process.env.STORAGE_BUCKET;
  if (!name) throw new Error('STORAGE_BUCKET is not set');
  return getStorage(app).bucket(name);
}
```

On App Hosting, `initializeApp()` with no arguments picks up the service account automatically.

5.3 Sign upload links

This route signs a link for one upload of an exact size. Build the file's path on the server, from the signed-in user, never from what the client sends.

```
// app/api/uploads/route.ts
import { NextResponse } from 'next/server';
import { bucket } from '@/lib/storage';

const MAX_BYTES = 200 * 1024 * 1024;

export async function POST(request: Request) {
  if (!process.env.STORAGE_BUCKET) {
    return NextResponse.json({ error: 'Storage is not configured' }, { status: 503 });
  }
  const uid = await requireUser(request); // your own sign-in check
  const { sha256, size } = await request.json();

  const validHash = /^[0-9a-f]{64}$/.test(sha256);
  const validSize = Number.isInteger(size) && size > 0 && size <= MAX_BYTES;
  if (!validHash || !validSize) {
    return NextResponse.json({ error: 'Invalid upload' }, { status: 400 });
  }

  const [url] = await bucket()
    .file(`users/${uid}/${sha256}`)
    .getSignedUrl({
      version: 'v4',
      action: 'write',
      expires: Date.now() + 15 * 60 * 1000,
      contentType: 'application/octet-stream',
      extensionHeaders: { 'x-goog-content-length-range': `${size},${size}` },
    });

  return NextResponse.json({ url });
}
```

- **The `503` check** answers clearly when the bucket isn't configured, instead of failing with a vague error. It also gives you an easy test after deploying (5.7).
- **`x-goog-content-length-range`** makes Cloud Storage refuse the upload unless it's exactly `size` bytes. The format is `min,max`, and here both are the same.
- **`expires`** can be at most 7 days away for v4 links. Keep it short.
- **The file name is the content's SHA-256 hash.** That's handy for spotting duplicates, but Cloud Storage doesn't check that the bytes match the name. Section 5.5 checks it on download.

5.4 Upload from the client

The client has to send **exactly** the headers the link was signed with: the same `Content-Type`, and the same length range. Anything different fails with a signature error.

```
const response = await fetch(url, {
  method: 'PUT',
  headers: {
    'Content-Type': 'application/octet-stream',
    'x-goog-content-length-range': `${file.size},${file.size}`,
  },
  body: file,
});
if (!response.ok) throw new Error(`Upload failed: ${response.status}`);
```

5.5 Sign download links, and check what arrives

Signing a download link is the same call, with `action: 'read'`:

```
const [url] = await bucket()
  .file(`users/${uid}/${sha256}`)
  .getSignedUrl({
    version: 'v4',
    action: 'read',
    expires: Date.now() + 5 * 60 * 1000,
  });
```

On the client, check the file against its hash before using it:

```
const data = await (await fetch(url)).arrayBuffer();
const digest = await crypto.subtle.digest('SHA-256', data);
const hex = Array.from(new Uint8Array(digest), (b) => b.toString(16).padStart(2, '0')).join('');
if (hex !== expectedSha256) throw new Error('Download does not match its hash');
```

5.6 Uploading from a web browser

A desktop app can use signed links as they are. A **web page** needs one more step: the browser checks with the bucket before a cross-origin upload, and a bucket with no CORS rules refuses. Desktop apps don't do this check.

1. Save this as `cors.json`:

   ```
   [
     {
       "origin": ["https://<your-domain>"],
       "method": ["PUT", "GET"],
       "responseHeader": ["Content-Type", "x-goog-content-length-range"],
       "maxAgeSeconds": 3600
     }
   ]
   ```

2. Open **Cloud Shell** (the terminal icon at the top right of the Cloud console), upload the file, and run:

   ```
   gcloud storage buckets update gs://<bucket-name> --cors-file=cors.json
   ```

List only your own site in `origin`. A `*` would let any website use links that leak to it.

5.7 Check it works after deploying

Deploy the site, then test it in two stages.

1. **Check the setting reached the site.** Call the upload route without signing in:

   ```
   curl -i -X POST https://<your-domain>/api/uploads
   ```

   - `503` means the site doesn't see `STORAGE_BUCKET`. Check `apphosting.yaml`, and that the new version has finished rolling out.
   - `401`, or whatever your sign-in check returns, means the setting arrived and the request got as far as sign-in.

2. **Check the bucket really works.** Step 1 doesn't prove the site can use the bucket or sign links. Only a real upload does, so sign in, upload a file through your app, then download it again. Then check it appears under **Cloud Storage** ▸ **Buckets** ▸ your bucket ▸ **Objects**.

---

6. Limits That Break Large Uploads

6.1 App Hosting refuses bodies over 32 MiB

App Hosting runs on Cloud Run, which refuses any request body over **32 MiB** (on the HTTP/1 servers sites normally use). A route that accepts uploads directly works in testing and then fails for big files. **Send large files straight to the bucket with a signed link,** never through the site.

6.2 Next.js's proxy keeps only the first 10 MB

In Next.js 16, the old `middleware` is called `proxy` (`proxy.ts`). The proxy buffers the body of every request it handles, and by default it keeps only the **first 10 MB**. The request doesn't fail: the route just receives part of the body, and the only sign is a warning in the server log. That's an easy way to end up with corrupt files.

If any route does receive large bodies, do both of these:

1. Raise the limit in `next.config.ts`:

   ```
   import type { NextConfig } from 'next';

   const nextConfig: NextConfig = {
     experimental: {
       proxyClientMaxBodySize: '50mb',
     },
   };

   export default nextConfig;
   ```

2. Keep large-upload routes out of the proxy entirely, with its `matcher`:

```
// proxy.ts
export const config = {
  matcher: ['/((?!api/dev-upload|_next/static|_next/image|favicon.ico).*)'],
};
```

6.3 The Storage emulator can't sign links

The Firebase Storage emulator handles ordinary file operations, but signing links isn't among them, so `getSignedUrl` won't give you a working link in development.

A workable pattern:

1. In development only, have your signing route return a link to **your own dev-only route** instead, such as `/api/dev-upload`, with a short-lived token you sign yourself.
2. That route checks the token, then writes the body to the emulator with `bucket().file(path).save(...)`.
3. Make it impossible to reach in production. Start the route with:

```
if (process.env.NODE_ENV === 'production' || !process.env.FIREBASE_STORAGE_EMULATOR_HOST) {
  return new Response('Not found', { status: 404 });
}
```

This route takes the file's body, so it's the one to exclude from the proxy (6.2).

---

7. Costs to Watch

Check current prices on Google's Cloud Storage pricing page. These are what you'll be charged for:

Charge	What drives it	How to keep it down
Storage	Every GB stored, per month	Delete files your app no longer needs
Operations	Each upload, download and listing	Avoid listing the bucket often. Store what you need in your database
Internet egress	Every GB downloaded by users	Downloads are usually the biggest cost. Don't re-download files users already have
Soft-deleted files	Deleted files kept for the soft delete period	Turn soft delete off (3.5)
Replication	Every GB written to a multi-region bucket	Use a single region (3.2)

The Blaze plan also lets you set a **budget alert** under **Billing ▸ Budgets & alerts**. It emails you as spending passes each threshold. It doesn't stop spending.

---

8. Troubleshooting

What you see	Why	What to do
The project is **missing** from the Cloud console	The console is signed in to a different Google account, or the picker is on **Recent**	Open the address in Section 2 and switch accounts. Use the **All** tab
The project picker doesn't show your **project ID**	The console shows display names, not IDs	Look for your project's name (`<project name>`) under the **All** tab
The form's summary says **Public access prevention: On**, but the bucket isn't protected	The summary doesn't match the unticked checkbox inside	Open the step and tick **Enforce public access prevention on this bucket** (3.4)
The new bucket is **multi-region**	The form starts on the US multi-region	Location can't easily be changed. If the bucket is empty, delete it and create it again in your region (3.2)
`curl` answers `NoSuchBucket`	The bucket name is wrong, or it wasn't created	Check the name under **Cloud Storage ▸ Buckets**
The upload route still answers `503` after deploying	The site doesn't see `STORAGE_BUCKET`	Check `apphosting.yaml` (5.1), and that the rollout finished (5.7)
Signing fails with an error mentioning `iam.serviceAccounts.signBlob`	The account can't sign for itself yet	Grant Token Creator on the account itself (4.2), enable the API (4.3), and wait a few minutes (4.4)
Upload fails with `SignatureDoesNotMatch`	The client's headers differ from the signed ones	Send exactly the signed `Content-Type` and `x-goog-content-length-range` (5.4)
Upload fails with **400**	The file's size is outside the signed length range	Sign the link with the file's exact size
Upload fails with `AccessDenied`	The service account can't write to the bucket, or the link expired	Grant Storage Object Admin on the bucket (4.1). Sign a fresh link
Browser upload fails with a **CORS error**	The bucket has no CORS rule for your site	Add one (5.6)
A route rejects big uploads, or they **fail only in production**	The body is over App Hosting's 32 MiB limit	Upload straight to the bucket with a signed link (6.1)
Large uploads arrive **cut off at 10 MB**, with a warning in the log	The proxy kept only the first 10 MB	Raise `proxyClientMaxBodySize` and exclude the route from the proxy (6.2)
`getSignedUrl` fails **only in development**	The Storage emulator can't sign links	Use a dev-only upload route (6.3)
The bill counts **deleted files**	Soft delete is on	Set its duration to 0 (3.5)
A bucket setting **can't be changed**	Location and hierarchical namespace are fixed once the bucket exists	Create a new bucket with the right settings and copy files across

9. Sources

Checked against these pages in November 2026. Menus move from time to time; if one looks different, the page will show its current name.

Google Cloud Storage

- Bucket locations
- Storage classes (availability SLAs)
- Availability and durability
- Hierarchical namespace
- Public access prevention
- Soft delete
- Signed URLs
- XML API headers (`x-goog-content-length-range`)
- Cross-origin resource sharing (CORS)
- Cloud Storage pricing

Google Cloud IAM and Cloud Run

- Service account permissions (Token Creator and `signBlob`)
- Create short-lived service account credentials
- Cloud Run quotas and limits (request size)

Firebase

- About App Hosting (the backend's service account)
- Configure App Hosting (`apphosting.yaml` environment variables)
- Connect your app to the Cloud Storage emulator

Node.js client library

- `File.getSignedUrl`

Next.js

- `proxyClientMaxBodySize`
- Proxy (formerly middleware)
- Upgrading to version 16

Latest version: github.com/ImmuneMoon/Private-Cloud-Storage-for-Firebase-App-Hosting

If this helped, you can support Fulllion Creative Works through PayPal or Ko-fi.

© 2026 Fulllion Creative Works. All rights reserved.